

UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master in Data Science and Business Informatics

DATA MINING I

Board Games Dataset Project

Report

Students:
Diprima Riccardo (715425)
La Conca Alessia (715358)
Torre Valerio (583569)

Course:
Data Mining

ACADEMIC YEAR 2025/2026

Contents

1 Data Understanding and Preparation **2**
- 1.1 Data Semantics . . . 2
- 1.2 Data Exploration . . . 3
- 1.3 Data Preparation . . . 4
  - 1.3.1 Data cleaning . . . 4
  - 1.3.2 Outliers Management . . . 4
  - 1.3.3 Missing Values Management . . . 5
  - 1.3.4 Feature Engineering . . . 5
- 1.4 Data Transformation . . . 5
- 1.5 Correlation Analysis . . . 6

2 Clustering **7**
- 2.1 Centroid-based Clustering . . . 7
  - 2.1.1 K-Means . . . 7
  - 2.1.2 Bisecting K-Means . . . 8
- 2.2 Hierarchical Clustering . . . 9
- 2.3 Density-based Clustering . . . 10
  - 2.3.1 DBSCAN . . . 10
- 2.4 General Results . . . 11

3 Classification and Regression **12**
- 3.1 Introduction . . . 12
- 3.2 Classification . . . 12
  - 3.2.1 K-NN Classification . . . 12
  - 3.2.2 Naive-Bayes Classification . . . 13
  - 3.2.3 Decision Tree Classification . . . 14
  - 3.2.4 Final Remarks . . . 15
- 3.3 Regression . . . 16
  - 3.3.1 Linear Regression . . . 16
  - 3.3.2 Lasso & Ridge Regression . . . 17
  - 3.3.3 K-NN Regression . . . 17
  - 3.3.4 Decision Tree Regression . . . 17
  - 3.3.5 Results . . . 17

4 Pattern Mining **18**
- 4.1 Parameters Selection . . . 18
- 4.2 Pattern Analysis . . . 19
  - 4.2.1 Frequent and Closed Itemsets . . . 19
  - 4.2.2 Maximal Pattern . . . 19
- 4.3 Association Rules . . . 20

1 Data Understanding and Preparation

1.1 Data Semantics

The dataset analyzed in this study comprises board game data sourced from a prominent online board game community. It includes information and ratings for a wide variety of games released over several years. The dataset contains 21,925 instances and 46 features. These features cover game difficulty, specs, user reviews, and categories. These features are broadly classified into categorical and numerical variables. Listed in Table 1 are some of the most important variables of the dataset with their type and description, considered the most significant for describing game mechanics and community reception.

Table 1: Description of Analyzed Variables

Variable	**Type**	**Description**
`BGGId`	Qualitative (Nominal)	Unique identifier for the game on BoardGameGeek.
`Name`	Qualitative (Nominal)	The official title of the board game.
`YearPublished`	Quantitative (Discrete)	Year of publication.
`MinPlayers`	Quantitative (Discrete)	Minimum number of players required.
`MaxPlayers`	Quantitative (Discrete)	Maximum number of players supported.
`MfgPlaytime`	Quantitative (Discrete)	Average playtime declared by the manufacturer (mins).
`ComMinPlaytime`	Quantitative (Discrete)	Minimum playtime estimated by the community.
`MfgAgeRec`	Quantitative (Discrete)	Minimum age recommended by the manufacturer.
`ComAgeRec`	Quantitative (Continuous)	Minimum age recommended by the community (averaged).
`LanguageEase`	Quantitative (Continuous)	Level of language dependence (amount of in-game text).
`GameWeight`	Quantitative (Continuous)	Complexity rating (scale 1-5).
`Rating`	Qualitative (Ordinal)	Community game rating (3 levels).
`NumOwned`	Quantitative (Discrete)	Number of users who own the game.
`NumWant` / `Wish`	Quantitative (Discrete)	Number of users who want or wish for the game.
`NumUserRatings`	Quantitative (Discrete)	Total number of user ratings received.
`NumExpansions`	Quantitative (Discrete)	Number of expansion packs released for the game.
`Kickstarted`	Qualitative (Binary)	Indicator if the game was funded via Kickstarter (1 if yes, 0 otherwise).
`Cat:[Type]`	Qualitative (Binary)	Dummy variables indicating the game category (e.g., *Strategy*, *Thematic*, *War*, *Party*).
`Rank:[Type]`	Quantitative (Discrete)	Current rank of the game in specific subdomains (e.g., *Strategy*, *Family*, *Abstracts*).

1.2 Data Exploration

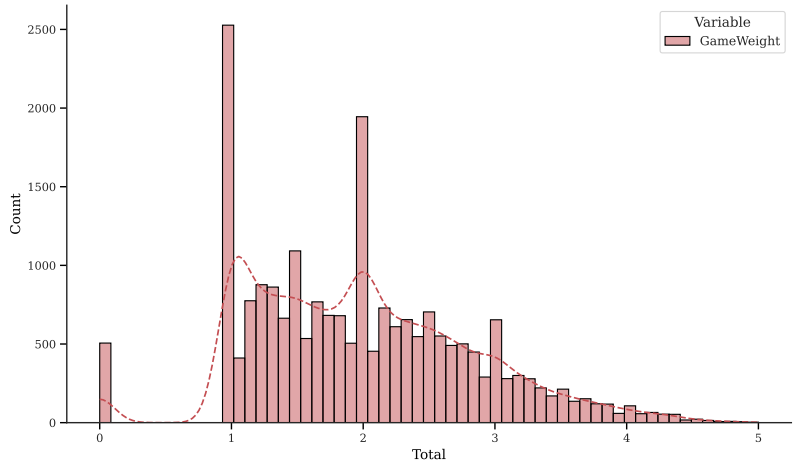
To gain insights into the dataset's characteristics, the main descriptive statistics (mean, standard deviation, quartiles) were computed for the key numerical features. Table 2 summarizes these values, providing an overview of the data range and central tendencies.

Table 2: Base Statistics of the Quantitative Variables

Variable	Mean	Std. Dev.	CV	Min	25% (Q1)	50% (Med)	75% (Q3)	Max
YearPublished	1985.50	212.49	0.11	-3500	2001	2011	2017	2021
MinPlayers	2.01	0.69	0.34	0	2	2	2	10
MaxPlayers	5.71	15.01	0.38	0	4	4	6	999
GameWeight	1.98	0.85	0.43	0	1.33	1.97	2.53	5
ComWeight	2.18	0.86	0.39	0	1.51	2.13	2.74	21
MfgAgeRec	9.61	3.64	0.38	0	8	10	12	25
ComAgeRec	10.00	3.27	0.33	2	8	10	12	21
LangEase	216.46	236.60	1.09	1	24.03	138	351	1757
MfgPlaytime	90.51	529.66	5.85	0	25	45	90	60000
ComMinPlaytime	63.68	443.92	6.97	0	20	30	60	60000
ComMaxPlaytime	90.51	529.66	5.85	0	25	45	90	60000
NumOwned	1499.58	5364.91	3.58	2	2	152	327	166497
NumUser Ratings	861.67	3638.68	4.22	30	56	123	395	108101
NumWant	41.69	117.26	2.81	0	3	9	28	2031
NumWish	228.46	788.48	3.45	0	14	39	127	19182
NumWeight Votes	49.48	205.76	4.16	0	4	9	26	7673
NumComments	0.00	0.00	–	0	0	0	0	0
NumExpansions	1.38	7.70	5.58	0	0	0	1	525
NumImplementations	0.31	0.85	2.75	0	0	0	0	38
NumAlternates	1.60	9.62	6.00	0	0	0	1	850

The analysis of these statistics reveals a few patterns in the data and unexpected results.
The first variable taken into account is *GameWeight*, a crucial feature throughout the various analysis performed. With a mean value of 1.98 and a low coefficient of variation (CV), it is a robust variable with a balanced distribution. In Figure 1 it is possible to observe a coupe of peaks where the value is an integer, which may be caused by either approximation or a low number of votes for a given game; there are over 500 games with a value of 0, outside the range 1-5, that will be dealt with. *GameWeight* values are very similar to the ones of the variable *ComWeight*, and this topic will be tackled in paragraph 1.3.

Figure 1: GameWeight distribution



Then, considering the variable *Number* (abbreviated to Num in table 2), their CV values are quite high, and their high maximum values, compared to the quartiles, suggest the presence of a heavy right-side tail in the data;

furthermore, the variable *NumComments*, is made of only 0.
A similar skewness was also found in the *Playtime* variables, with high CV and Maximum; the features are quite similar, with *MfgPlaytime* and *ComMaxPlaytime* being identical. These problems will be addressed in paragraphs 1.3 and 1.4.
Lastly, it is worth noting that the two *Recommended Age* variables are really similar and, as it will be seen afterward, strongly correlated.

1.3 Data Preparation

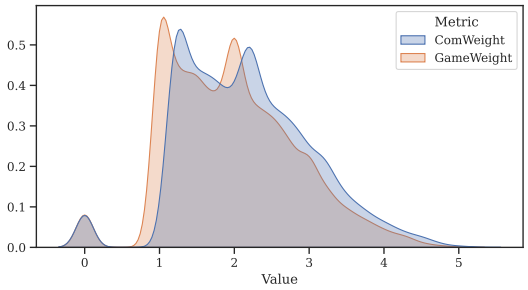
After conducting a first exploration, it was revealed that many variables in the dataset present missing values, null values, off scale values and outliers.For this reason, an in depth data cleaning phase was needed to prepare the dataset for further analysis.

1.3.1 Data cleaning

During the preprocessing phase, a feature selection process was performed to identify the most informative variables. Attributes considered irrelevant or redundant were removed from the dataset to improve the analysis.

- **Removed features:**
  - *ComWeight*: Removed, as anticipated, due to extremely high correlation with *GameWeight* (0.997); both variables give similar information regarding game complexity. In figure 2 it is shown the extremely correlated distributions of the two variables.
  - *BestPlayers* - *GoodPlayers* : Excluded due to high sparsity (predominance of zero values).
  - *ComMaxPlaytime*: Resulted in a duplicate of *MfgPlayTime*, which was selected as the reference variable.
  - *NumComments*: Removed due to a total lack of values.
  - *ImagePath* - *Description* - *Name* : Descriptive attributes dropped since they were not needed for this analysis (*BGGId* is sufficient for identification).

Figure 2: Distribution of ComWeight vs GameWeight



1.3.2 Outliers Management

To mitigate the impact of outliers, specific upper thresholds were defined based on a combination of domain knowledge and statistical analysis. The *MaxPlayers* variable was restricted to a maximum of 99, while playtime metrics—specifically *MfgPlaytime*, *ComMinPlaytime*, and *AveragePlaytime*—were limited to 240 minutes, a value aligned with the 95th percentile of *MfgPlaytime*. Furthermore, an upper bound of 841 was applied to *LanguageEase*, determined using the standard interquartile range formula ($Q_3 + 1.5 \times \text{IQR}$). All values exceeding these thresholds were capped at the threshold's value to ensure greater robustness of the analysis without losing more information. It was decided not to cut any of the *Number* variables; the outliers of those variables are simply the most popular board games (such as Monopoly and Risk), and the high values are not errors but real metrics.

1.3.3 Missing Values Management

Subsequently, data cleaning was carried out by handling null and missing values. To ensure rigorous evaluation and prevent data leakage, the dataset was first split into Training (80%) and Test (20%) sets. A stratified split based on the Rating variable was employed to preserve the target distribution. After the split, missing values were handled using two distinct strategies depending on the variable type.
Regarding null values, the quantitative variables related to gameplay time, namely *MfgPlaytime* and *ComMinPlaytime*, were handled via Mode Imputation, a strategy chosen to preserve the original data distribution.
Conversely, missing values for *GameWeight*, *ComAgeRec*, and *LanguageEase* were managed using Iterative Imputation. This multivariate method models each feature with missing values as a function of the others. For this purpose, a Random Forest Regressor ($n_estimators = 22$) was selected as the estimator. This model handles non-linear relationships and outliers well. Furthermore, since it is invariant to monotonic transformations, feature scaling was not necessary. Crucially, the imputer was fitted only on the training set and then applied to the test set to avoid data leakage. The process ran for 10 iterations, starting from the median.
For every stochastic operation performed in the study a $randomseed = 222$ was used to ensure reproducibility.

1.3.4 Feature Engineering

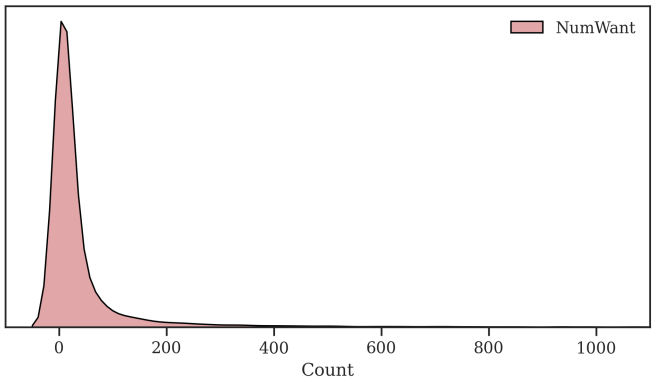
In this phase, new derived variables were created to improve the dataset.

- Period: Starting from the variable *YearPublished*, the column *Period* was created to group games into meaningful time periods.
- Age: Calculated as the difference between the current year and *YearPublished*, to represent the chronological age of the game.
- Average Playtime: Calculated as the arithmetic mean between the minimum and maximum playtime, providing a representative value of the duration of a game session. This variable was used as a substitute for the other two features, that, due to high correlation, could not be used together in the analysis.

1.4 Data Transformation

To ensure the effectiveness of subsequent statistical analyses and improve the performance of distance-based algorithms, the distribution of numerical variables was examined. Specifically, the skewness of each feature was calculated to identify significant asymmetries. The most skewed features were the *Number* ones; in Figure 3, it is possible to observe the extremely long tail of such distributions. A threshold of |1.0| was set to detect highly skewed variables; of all the quantitative variables, 13 of them were above the threshold. For those features an appropriate transformation was applied depending on the type of data. A logarithmic transformation ($\log(1 + x)$) was applied to all the variables except one; the function that was used accounts for values equal or greater than 0, but cannot handle negative numbers. Transformed variables are: *MinPlayers, MaxPlayers, NumOwned, NumWant, NumWish, NumWeightVotes, NumUserRatings, NumAlternates, NumExpansions, NumImplementations, Age, AveragePlaytime*. The only feature left out is YearPublished, that contains negative values as low as -3500. A cube root transformation was used instead. Unlike the logarithmic transformation, the cube root is defined for negative values, effectively handling historical dates (e.g., years B.C.) while reducing skewness.

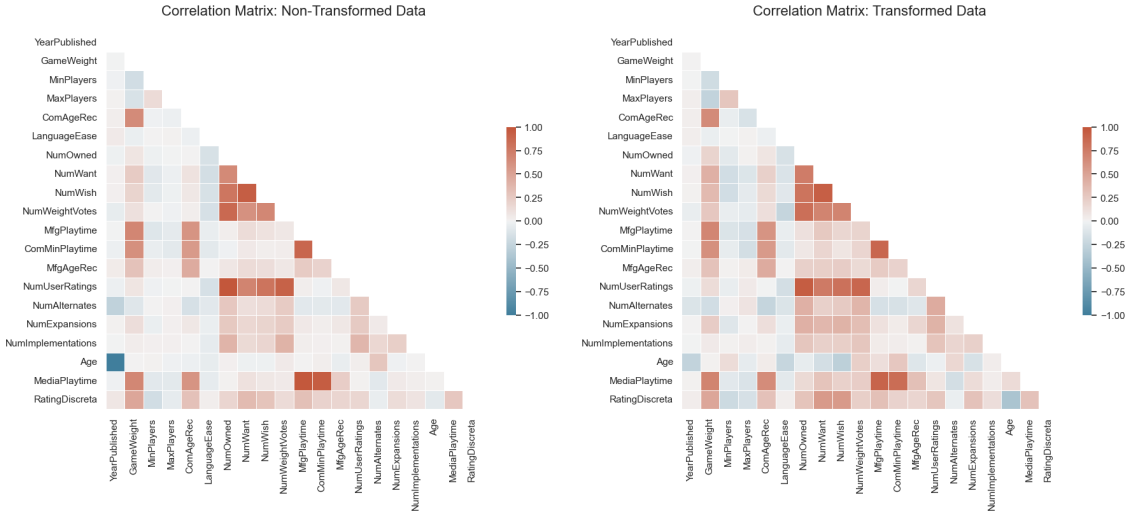
Figure 3: NumWant Distribution



1.5 Correlation Analysis

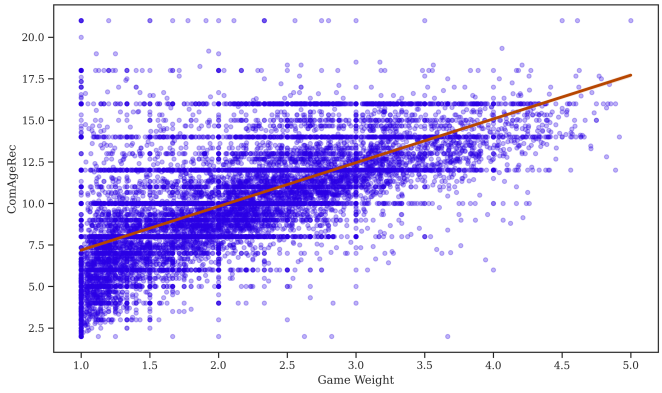
A correlation analysis was conducted to examine linear relationships and detect potential multicollinearity. Although clustering and tree-based models largely utilized the original dataset, the analysis of the transformed set was crucial for identifying underlying linear dependencies that might be masked by skewness in the raw data. This provided a clearer view of potential redundancy among features. Prior to the calculation, the categorical variables *Rating*, (*BGGId*, *Family*, *Period*, *Rec_Year*), were excluded due to their qualitative nature.

Figure 4: Correlation Matrices



As illustrated in Figure 4, the heatmap correlation matrix highlights several key insights regarding the dataset structure. A very strong positive correlation (near +1.0) is observed among *Number* variables related to user activity, such as *NumOwned*, *NumWant*, *NumWish*, and *NumUserRatings*. This values suggest that the features carry similar information: popular games tend to score high across all engagement dimensions simultaneously. Yet, the correlation values were not high enough to remove any of those variables from the dataset.
GameWeight showed moderate correlations with a few variables, such as *Average Playtime* and *Com. Age Rec.*, indicating that complex games often require longer sessions and older players. Figure 5 illustrates the scatter plot of *GameWeight* and *ComAgeRec* that, with a Pearson's Correlation of 0.66, indicates a moderately strong positive association between the two variables.

Figure 5: Scatter plot of ComAgeRec and GameWeigh



2 Clustering

The clustering section aims to discover patterns among the specific group of features selected by the team. To avoid the curse of dimensionality, five variables were selected: *AveragePlaytime, GameWeight, ComAgeRec, MinPlayers, YearPublished*. For visual representation Principal Component Analysis (PCA) was employed. For every clustering method the attributes were scaled with the Min-Max Scaler, chosen over the Standard scaler. This selection was made because the Min-Max Scaler provided better results in terms of cluster separation and Silhouette Scores.

2.1 Centroid-based Clustering

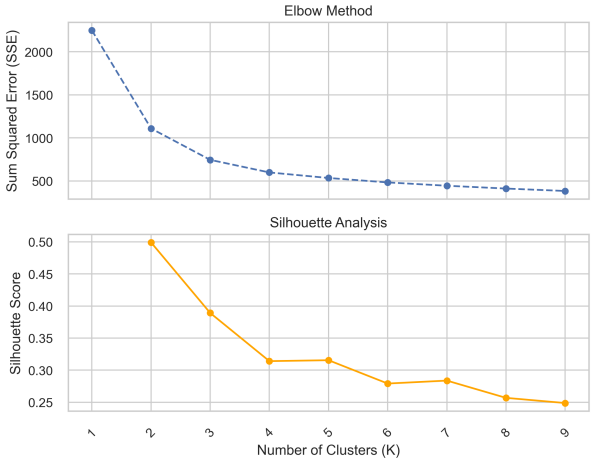
Centroid-based clustering is a category of clustering algorithm where each cluster is associated with a central vector, the centroid; the various data points are assigned to the group with the closest center.
For the centroid-based clustering, two methods were analyzed: K-Means and Bisecting K-Means. The analysis involved testing the algorithms on both the standard and transformed feature sets. After analysing the results the team chose the non-transformed data as they gave better overall results. Regarding PCA, the first component represents 74.14% of the variance, while the second 12.96%.

2.1.1 K-Means

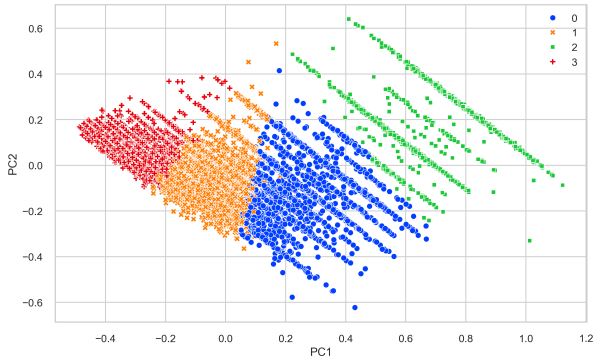
K-Means is a centroid-based clustering method that randomly chooses the initial centroids. The optimal number of clusters for the K-Means method was determined to be $k = 4$. This choice is consistent with the Elbow Method and effectively identifies four distinct groups that characterize the rating distribution across the clusters. The number of times the algorithm was run is 10, and for each run the maximum number of iterations was 100. The corresponding Silhouette and SSE (Sum Squared Error) values are 0.31 and 599.12.

Figure 6: K-Means Clustering

(a) SSE and Silhouette



(b) Clustering Visualization



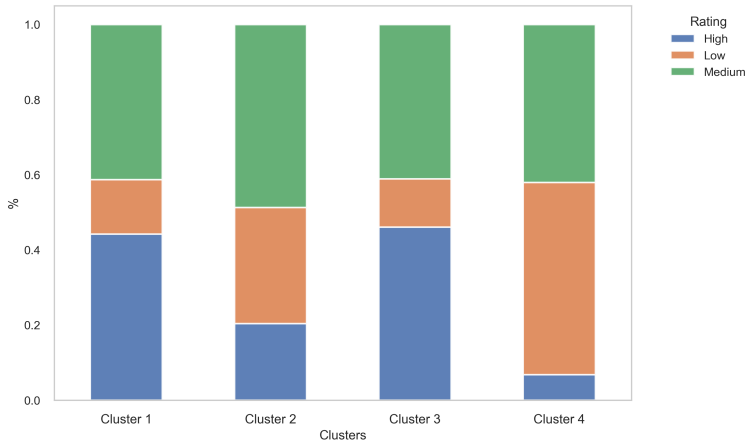
The 4 identified clusters were characterized and labeled by analyzing the attribute values of their respective centroids as detailed in Table 3. While there is little variation between the centroids for the feature *MinPlayers*, even slight differences are meaningful: groups 1 and 3, the clusters with the most complex games, include most of the titles that support a single-player mode. Similarly, the differences in the *Year* variable indicates that, in the past, games tended to be more complex and longer to play out. For these aspects, the team decided to keep said variables for the clustering analysis.

Table 3: Cluster Centroids Characterization

Cluster Label	PlayTime	Weight	AgeRec	MinPlayers	Year	Description
1: Mid-Range	86.14	2.87	12.56	1.83	2005.29	Games targeting a teen/adult audience and the core hobbyist market. Single player oriented.
2: Family Mid-Range	42.95	1.94	10.31	2.08	2004.94	Accessible titles with moderate duration and complexity. Ideal for families and casual gamers.
3: Expert Long-Play	213.07	3.21	14.03	1.89	2000.09	Older games with very long duration and high complexity. Targeted at hardcore enthusiasts.
4: Entry Level	24.69	1.26	6.83	2.07	2003.01	Short fillers with minimal rules complexity. Suitable for young children.

Figure 7 illustrates the distribution of the target variable *Rating* across the identified clusters. *Rating* was not included in the clustering feature set to observe its distribution for external validation of the groups.
In all the clusters the most common *Rating* is Medium. Cluster 3 (Expert Long-Play) exhibits the highest proportion of high ratings, consistent with the tendency of hobbyist communities to rate complex games more favorably. Conversely, Cluster 4 (Entry Level) and Cluster 2 (Family) show a more heterogeneous rating distribution, with Cluster 4 having the highest percentage of low ratings, further confirming the pattern observed in Cluster 3, where rating tends to scale with complexity.

Figure 7: Rating Distribution across Clusters

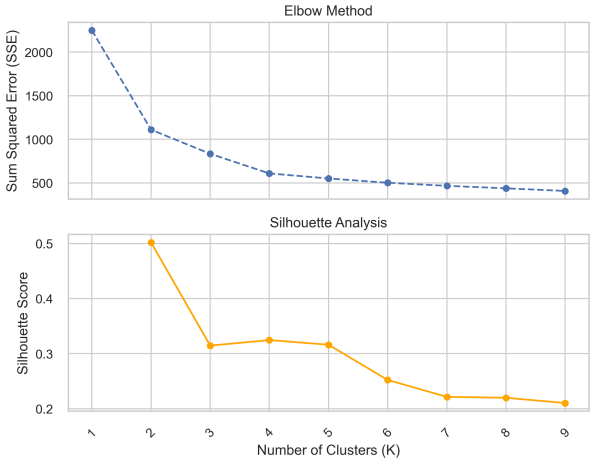


2.1.2 Bisecting K-Means

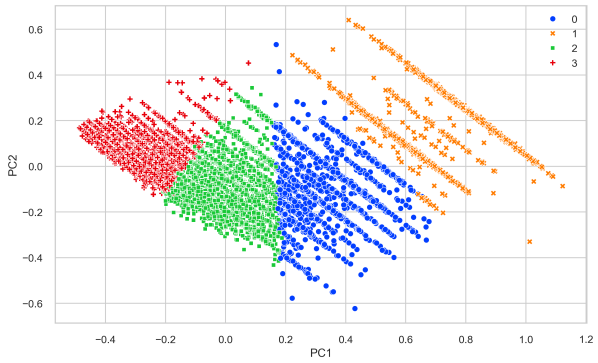
Bisecting K-Means is a variant of K-Means that can produce a hierarchical clustering and does not use any criterion for stopping bisections before singleton clusters are reached. Following the same methodology applied to K-Means, the value of k has been selected analysing SSE and Silhouette values. The team selected $k = 4$ following the Elbow Method for SSE and as it presented the highest Silhouette value in a stable values within $k = 3, 4, 5$ (excluding the trivial solution of $k = 2$, which was discarded a priori as it would result in a too generic binary segmentation). The algorithm was run utilising the same values of K-Means for number of iterations and different initial centroids. Silhouette and SSE values resulted in 0.32 and 407.48 respectively.

Figure 8: Bisecting K-Means Clustering

(a) SSE e Silhouette



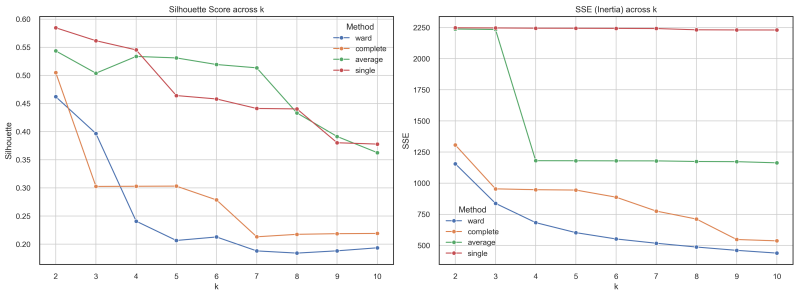
(b) Clustering Visualization



2.2 Hierarchical Clustering

Hierarchical clustering is an unsupervised technique that builds a nested structure of groups by treating every point as a separated cluster, and iteratively merging the two closest ones until all the data is unified into one single cluster. For the Hierarchical Clustering analysis, the Ward, Complete, Average, and Single linkage methods were evaluated. The four techniques only differ by the way the dissimilarity between clusters is calculated. For this method, the same dataset and PCA previously obtained with Centroid Clustering methods were used. By observing the SSE, dendrogram structures and cluster distributions, the team decided to concentrate on Complete and Ward Linkages, as they best represented a meaningful separation of the clusters. Conversely, the Single and Average methods were discarded due to the 'chaining' phenomenon, which resulted in highly unbalanced groups despite their Silhouette values, while the SSE values remained substantially flat, indicating an inability to effectively separate the data into distinct groups.

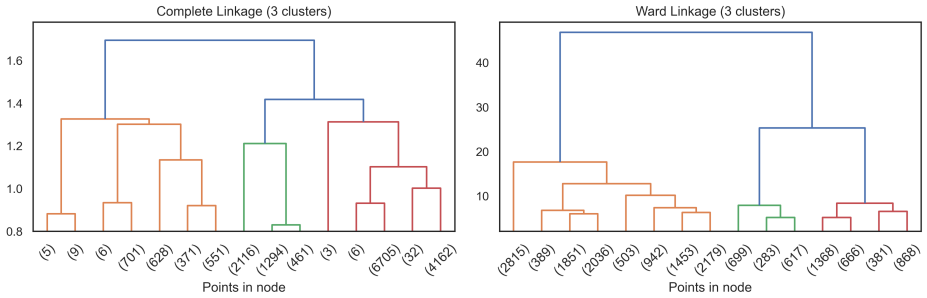
Figure 9: SSE and Silhouette Values



As illustrated in Figure 9, the SSE analysis for the Ward Linkage presents a distinct elbow at $k = 3$. A similar trend is observed for the Complete Linkage, where $k = 3$ was selected over $k = 5$, preferring the simplest model that provides comparable results.

Consequently, Figure 10 details the dendrograms for the selected methods. Ward Linkage divides the dataset into three macro-categories of comparable dimensions, with one still larger than the other two, while the Complete method reveals a distinct pattern. Specifically, Complete Linkage isolates a visibly smaller central cluster, likely representing a compact subgroup of outliers or highly specific games, at the expense of the homogeneity provided by Ward. This is coherent with their corresponding objectives: Ward tries to obtain a balanced distribution, while Complete Linkage allows for the isolation of smaller and more specific groups, being strictly sensitive to outliers and maximum distances.

Figure 10: Hierarchical Clustering



The results of Silhouette and SSE values are 0.30 and 954.49 for the Complete Linkage, 0.39 and 837.98 for the Ward Linkage

2.3 Density-based Clustering

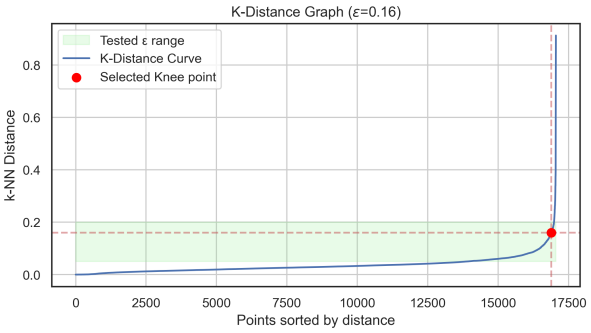
Density-based clustering is a method that finds clusters by looking for areas where data points are more concentrated. Unlike the other two methodologies, in density based clustering, points in low-density regions are classified as noise and omitted. The OPTICS algorithm did not show insightful results, leading the team to focus on DBSCAN. Unlike in center-based clustering, in density-based clustering transformed data revealed better results regarding Silhouette values and cluster separation. In this case, the first PCA component accounts for 67.26% of the variance while the second for 14.85%.

2.3.1 DBSCAN

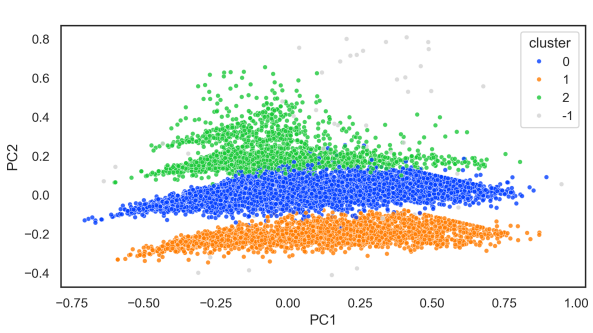
DBSCAN is a density-based clustering algorithm that produces a partitional clustering, in which the number of clusters is automatically determined by the algorithm. The partitioning is determined by the value of the parameter ε; the greater the value, the larger is the distance within which a point can identify its neighbours. The starting *MinPts* parameter was selected following the theoretical rule of thumb $MinPts \approx 2 \cdot D$ (where D is the number of variables), resulting in a value of 10. To optimize the model, a grid search was performed, testing *MinPts* values between 8 and 20 and ε values from 0.05 to 2.
The best parameters combination was identified at $MinPts = 10$ and $\varepsilon = 0.16$, generating a three clusters solution, effectively partitioning the data while isolating only 58 points as noise (0.34% of the total dataset).
From a Silhouette score perspective, DBSCAN failed to provide an optimal clustering solution for this dataset, resulting in a value of 0.072. This low value highlights the geometric structure of the data. The clusters appear more as layers than as separated globular shapes. Consequently, the Silhouette metric, penalizes the density-based clusters found by DBSCAN, since it assumes that clusters should be well-separated by empty space. Despite this, the algorithm achieved a clean separation of regions with a small number of noise points, correctly capturing the dataset structure.

Figure 11: DBSCAN

(a) Epsilon value



(b) Clustering Visualization



2.4 General Results

Following the detailed analysis of each clustering method, the results are summarized in Table 4. These values cannot be interpreted without context: by observing only the Silhouette scores, Single and Average Linkage might appear as the best performers. However, their inefficiency and high variance (SSE) have already been discussed in their respective sections.

Table 4: Silhouette and SSE Scores

Algorithm	**Silhouette**	**SSE**
K-means	0.31	534.78
Bisecting K-means	0.32	407.48
Dbscan	0.07	NaN
Hier: Complete	0.30	954.49
Hier: Single	0.54	2244.57
Hier: Average	0.53	1180.61
Hier: Ward	0.39	837.98

For general purposes, the Bisecting K-means shows the best compromise, achieving the lowest SSE value (407.48) combined with a reasonable Silhouette score. It is closely followed by standard K-Means. This suggests that the Bisecting K-Means approach effectively better captured the structure of the data than the random initialization of standard K-means. Among the hierarchical algorithms (excluding those affected by chaining), Ward Linkage achieves the highest Silhouette result, with a value of 0.39.
A general observation regarding the Silhouette scores shows that, excluding Single and Average results, all the other algorithms resulted in values between 0.30 and 0.40. These values indicate a structure that confirms the hypothesis raised during the DBSCAN analysis: the dataset is characterized by layers rather than well-separated globular clusters.

3 Classification and Regression

In this section, the analysis performed for regression and classification tasks will be discussed.

3.1 Introduction

Classification and regression represent two of the fundamental techniques of predictive modelling in supervised learning. Both methodologies share a common objective: to create a mapping function $f : \mathcal{X} \to \mathcal{Y}$ that is able to predict a target variable Y based on a set of input features X by minimizing a certain loss function of the prediction error. The main distinction between the techniques lies in the nature of the output space Y. On one hand, Classification is designed to predict the values of qualitative variables, such as the *Rating* variable in the dataset; on the other hand, Regression is best used when predicting continuous numerical variables. The following sections detail the specific models applied to these tasks and the evaluation metrics used to assess their performance. All the models were fitted to the train dataset and were then evaluated on the test dataset.

3.2 Classification

In the classification setting, the goal is to predict discrete class labels. The target variable Y can assume one of K different levels; if $K = 2$ the task is defined as a binary classification one, if $K > 2$ then the problem is defined as a multi-class classification task.

Feature Selection
During the team's analysis, *Rating* was chosen as the target variable Y. Thanks to the team's previous knowledge on the topic, the choice of the various X features was made by conducting a One-Way Analysis of Variance (ANOVA). In particular, each feature was tested to verify whether its mean value significantly changed across the three levels of *Rating*; afterward, the relevant features were ranked by calculating η^2, a metric that shows the proportion of variance in the feature that is explained by the target class.
After this preliminary analysis, the team chose as the X independent variables: *GameWeight, NumWant, MediaPlaytime, ComAgeRec, MinPlayers and NumUserRatings.*

Data Preparation Strategy

The team identified three classification methodologies to predict the *Rating* output variable: K-Nearest Neighbours (K-NN), Naive Bayes and Decision Tree classifiers.
Each of the models has different theoretical assumptions and need specific data configurations to ensure optimal performance. Three variations of the dataset were prepared to conduct the classification analysis. Since the team already performed the partitioning of the dataset during the data preparation phase, it was only necessary to decide whether to utilize the transformed or untransformed data. For the Decision Tree classification method the untransformed dataset was utilized; it was unnecessary to utilize the log-transformed data since the decision tree classifier algorithm bases its decision on the order of the values and not their distances. The Naive Bayes classifier assumes that the variables utilized for the classification are normally-distributed; considering the skewed nature of most variables in the study, it was necessary to utilize the transformed dataset. Finally, for the K-NN classifier it was necessary to use a transformed and scaled dataset; the logarithmic transformation is useful to reduce the outliers' effect on the model, while the standard scaling was mandatory to remove the effect of the different scales of the values in the dataset.
In the following subsections, every methodology will be shown in full details.

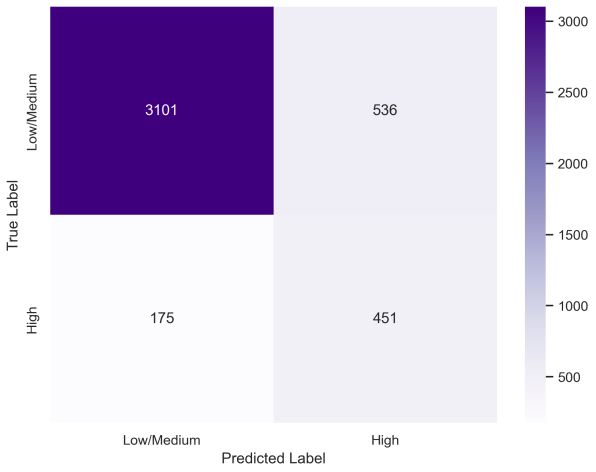
3.2.1 K-NN Classification

The K-NN classification algorithm is a non-parametric learning method. It classifies a new observation by finding its k nearest neighbours in the input space, and then assigning it a label through a majority voting system between the identified k observations.
The analysis was performed by configuring the K-NN classifier with the Euclidean distance metric and adopting a distance-weighted voting scheme. Initially a binary classification was implemented with *Rating* variable split in two categories: High and Low/Medium (see Figure 12a). The team felt that this split was too vague and decided to go on with a multi-class classification using all three Rating levels. The model was firstly trained using $k = \sqrt{n} \approx 131$, a general rule, with promising results; subsequently, an hyperparameter tuning phase was conducted to refine the

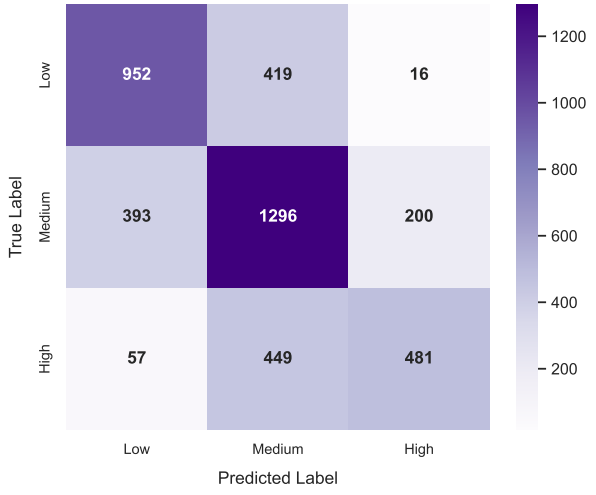
choice. A two-strategy phase utilizing 10-fold cross-validation was adopted. A preliminary research was performed over a broad range of k, from 10 to 200 with a step of 10; the region contained in the interval $[70, 120]$ was identified as the one containing the highest modest accuracy. A more granular research was made inside the identified interval, checking each possible k, to find the local maximum. After analyzing mean accuracy and deviation for the validation folds, the value $k = 71$ was found as the local (and likely global) accuracy maximum.
Figure 12b represents the confusion matrix obtained from the model. Overall, the main diagonal, that contains the true positives for every class, has more samples compared to the off-diagonal elements, which can be quantified with an accuracy rating of 0.640. However, it is possible to observe a significant number of misclassifications toward the *Medium* class column. There is a quite large number of true *High* and true *Low* samples, 449 and 419 respectively, that are classified as *Medium*, suggesting the presence of bias toward the center class.

Figure 12: Confusion Matrix

(a) Binary Classification



(b) Multiclass Classification



In Table 5 a few key metrics about the model are reported. The *High* class recall is particularly low, indicating that around 51% of the true *High* samples are misclassified; F1-Score, an indicator that combines precision and recall, is consequently also lower compared to the other two classes.

Table 5: Classification Report Results: K-NN Classifier

Class	Precision	Recall	F1-Score	Support
High	0.69	0.49	0.57	987
Low	0.68	0.69	0.68	1387
Medium	0.60	0.69	0.64	1889

3.2.2 Naive-Bayes Classification

The Naive Bayes classifier is a probabilistic machine learning model based on Bayes' Theorem. Given a class variable y and a feature vector $X = (x_1, \ldots, x_n)$, to label a new observation the classifier calculates the posterior probability and assigns the class with the highest value. For this analysis, there was no need to adjust the classifier's hyperparameters, and the function was used as is.
The model managed to obtain an accuracy score of 0.576, quite lower compared to the K-NN one. Observing Figure 13, the confusion matrix obtained from the model, the main diagonal is, again, more densely populated than the areas outside of it. Many true *Medium* samples have been misclassified, bringing the overall accuracy score down; in particular, 667 true *Medium* samples have been classified as *Low*, which means that the model is not able to correctly separate the two classes. The classifier is also biased toward the *Low* class, since over 1800 samples have been classified as such while the true *Low* class samples are 1387. In Table 6, there is a great difference when it comes to the recall values; while *Low* class can boast a value of 0.75 (the highest recall of the study), *Medium* class

presents a value of 0.45 (the lowest recall of the study), meaning around 55% of the true *Medium* class games have been misclassified.

Figure 13: Confusion Matrix: Naive-Bayes Classifier

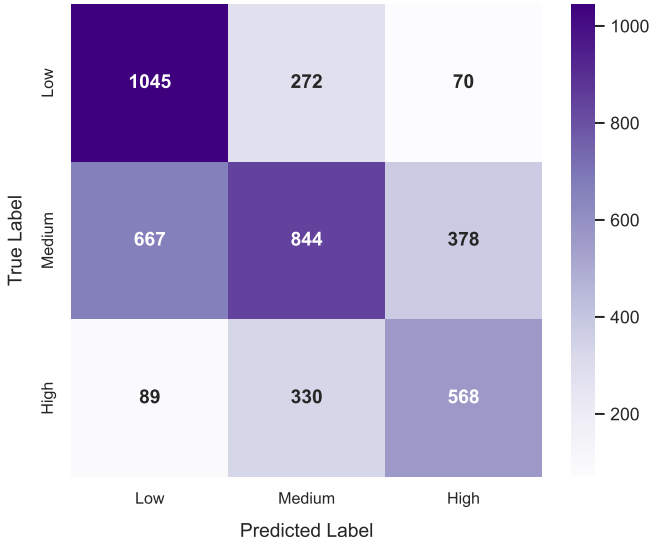


Table 6: Classification Report Results: Naive-Bayes Classifier

Class	Precision	Recall	F1-Score	Support
High	0.56	0.58	0.57	987
Low	0.58	0.75	0.66	1387
Medium	0.58	0.45	0.51	1889

3.2.3 Decision Tree Classification

The Decision Tree Classifier is a non-parametric supervised learning method based on simple decision rules. The model creates a hierarchical structure made of a root node, internal nodes and leaf nodes. Such tree is generated by a recursive partitioning process that, at each internal node, splits the dataset into subsets based on a certain attribute threshold.

It is essential to accurately choose the hyperparameters for a decision tree classifier, since this kind of models are prone to overfitting. For this reason, a grid search was carried out, testing: three depths from 1 to 20, minimum samples per leaf from 1 to 500 and minimum samples per split from 0.002 to 0.2. The best configuration of hyperparameters identified uses Gini Impurity as the plotting criterion, with a tree depth of 8, 100 minimum samples per leaf and 0.01 minimum samples per split.

The model obtained an accuracy score of 0.621, similarly to the K-NN classifier. Observing Figure 14, it appears that this model is once again biased towards the middle class *Medium*; many true *High* and *Low* class samples have been misclassified specifically as *Medium*, and very rarely as the non-adjacent class. Looking at Table 7, the values are quite similar to the ones from Table 5, and their interpretation remains the same. The final decision tree can be observed in Figure 15 and as a shortened version in Figure 16.

Figure 14: Confusion Matrix: Decision Tree Classifier

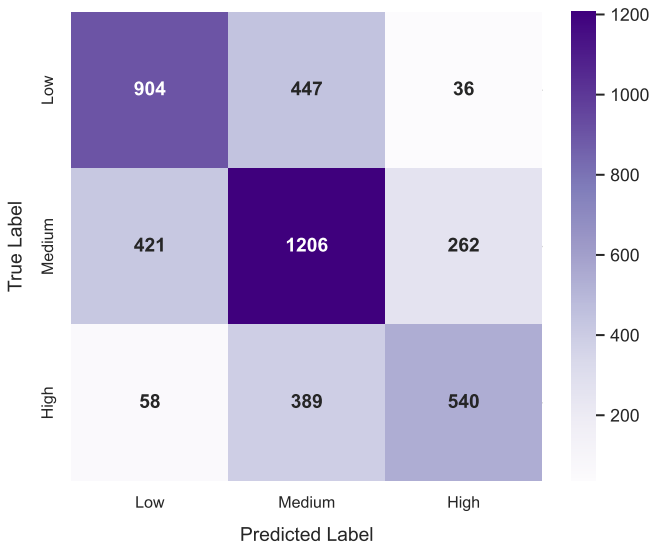


Table 7: Classification Report Results: Decision Tree Classifier

Class	Precision	Recall	F1-Score	Support
High	0.64	0.55	0.59	987
Low	0.65	0.65	0.65	1387
Medium	0.59	0.64	0.61	1889

Figure 15: Decision Tree: Complete

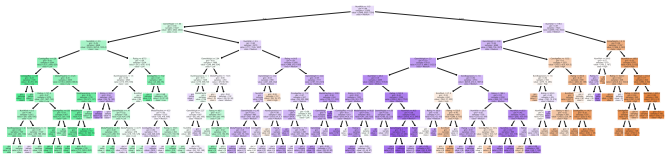
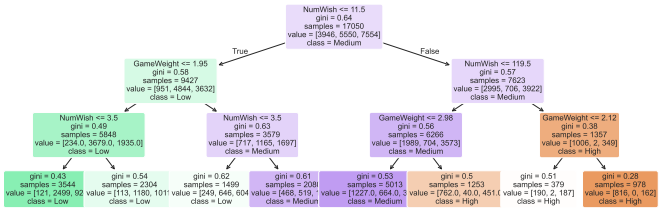


Figure 16: Decision Tree: Short

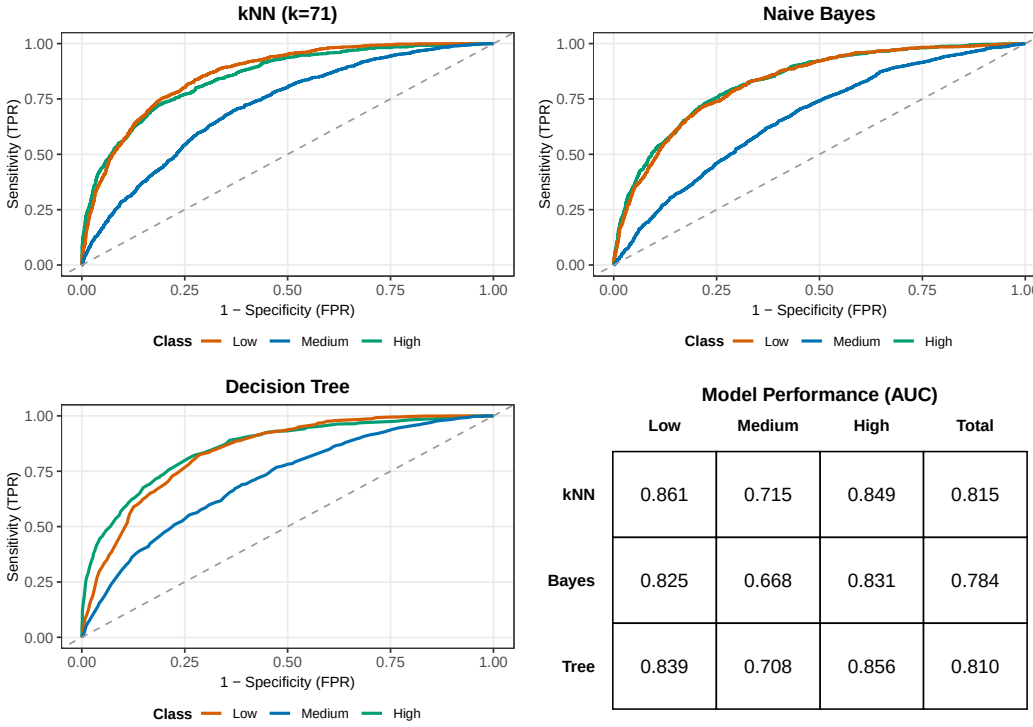


3.2.4 Final Remarks

Having analyzed each of the classifiers individually, it is now possible to proceed with a comparative assessment. In terms of accuracy, the K-NN classifier achieved the highest score of 0.640, followed by the Decision Tree and the Naive-Bayes classifiers with 0.621 and 0.576 respectively.
Yet, the accuracy score alone is not enough to determine which classifier is truly the best, especially considering the unbalanced nature of the three classes. For this reason, the Receiver Operating Characteristics (ROC) curves were calculated. Since the *Rating* variable possesses more than two classes, a One vs All (OvA) procedure was used to conduct the analysis. Afterward the Area Under the Curve (AUC) was calculated to determine the overall

ability of the model. In Figure 17, it is possible to observe both the various ROC curves and a comparative table containing the AUC values. For each of the models, the *Low* and *High* curves were by far the better ones, while the *Medium* curve was always a bit below the others. The classifiers struggle to distinguish the intermediate class from the extreme ones, as there is a higher degree of feature overlap between *Medium* and the other two classes. On the other hand, the models show a great performance when identifying the extreme classes.
Looking at the average AUC values (0.815) and the accuracy score (0.640), the K-NN classifier is confirmed as the best model.

Figure 17: Classifiers Comparison: ROC Curves



3.3 Regression

In the regression setting, the goal is to predict a continuous target variable Y utilizing a set of input features. The regression models study the relationship between the output variable and the regressors by fitting a function to the data. Such models can either be linear or non-linear. A linear regression model assumes the relationship between input and output to be linear in nature, and the resulting function can be described by a line or an hyperplane based on the number of independent features. Non-linear regression models, on the other hand, are able to handle more complex non-linear relationships between output and input; they are usually represented by curves or complex surfaces based on the number of independent variables.
In the analysis, a few techniques were implemented to understand the best predictive model for the data. The models used were linear, lasso, ridge, K-NN and decision tree. For the various linear regression models, the transformed dataset was used; for the K-NN regression model, it was necessary to use the transformed and scaled dataset; lastly, for the decision tree regression model the non-transformed dataset was enough. The motivations behind choosing a certain dataset for a model are analogous to the ones for the classification problem. The regressors chosen for the analysis were all the sixteen quantitative variables, and the chosen dependent variable was *AveragePlaytime*.

3.3.1 Linear Regression

Linear regression is widely considered the baseline modelling regression technique, studying the linear relationship between the dependent variable Y and the regressors X. To fit the model, the Ordinary Least Squares method is used; such procedure estimates the best parameters by minimizing the residual sum of squares. The resulting function is the hyperplane that best approximates the data.
At first, a simple regression model was created. The single chosen regressor was *GameWeight*, that had the

highest correlation with Y; the model achieved an R^2 of 0.498, which indicates that *GameWeight* plays a key role in explaining Y variability. Afterward, a multivariate model was fitted. To determine the best combination of regressors (mainly to avoid having to deal with an overly complex model), a bidirectional stepwise selection was used; the team previously knew of this technique and felt confident using it. Finally, the collinearity between features was checked through the analysis of VIF. The final model was based on seven features, and achieved an $AdjustedR^2$ of 0.574.

3.3.2 Lasso & Ridge Regression

Lasso and Ridge regression are both variations of the standard linear regression.
Lasso regression modifies the OLS function by adding a penalty to the sum of the absolute values of the coefficients; this new function depends on a λ parameter, and as the value gets bigger the model tends to eliminate the less useful features (similarly to the stepwise method). Ridge regression is another technique based on OLS that regularizes the cost function. The model modifies the function by adding a penalty to the sum of the squared coefficients. Just like lasso, ridge also depends on a λ value.
While lasso regression tends to create less complex models by eliminating less relevant features, ridge regression tends to retain all the features and instead shrinks the coefficients toward zero to mitigate overfitting.
The best λ were estimated by using a specific *sklearn* function. The best value for the lasso model is 0.01, while for the ridge model is 10. The two models achieved an $AdjustedR^2$ of 0.577 and 0.585, both slightly higher than the standard multivariate linear regression.

3.3.3 K-NN Regression

K-NN regression is a non-parametric algorithm part of the non-linear regression family; it assign a value of Y by calculating the mean value between of the K nearest neighbours.
Both a simple and a multivariate model were calculated. The simple model utilized *GameWeight* as the sole predictor; the best value of K, 117, was determined after a grid search. The model yielded an R^2 value of 0.504. For the multivariate model, a meticulous grid search was performed for both the best K and the most appropriate features to use through a stepwise procedure that minimizes MSE. Seven features were selected, and the optimal K was chosen as 50. The model produced an $AdjustedR^2$ of 0.627.

3.3.4 Decision Tree Regression

Finally, Decision Tree Regression is another non-linear regression model. The algorithm produces a tree structure that assigns a value to a new observation by calculating the average output value of the training samples that were assigned to the corresponding leaf node.
Both a simple and a multivariate model were calculated. For the simple model, a depth of 4 was selected after a grid search; the model achieved an R^2 of 0.476. The multivariate model was built considering a depth of 6 and fourteen features, the combination was found through a grid search. The model obtained an $AdjustedR^2$ of 0.594

3.3.5 Results

Having analyzed each of the regression models individually, it is now possible to proceed with a comparative assessment. Table 8 contains the main indicators of model's performance. As expected, the inclusion of multiple predictors leads to better results in each kind of model.
When considering multivariate linear models, Ridge Regression was the better performing one, suggesting that, in this scenario, handling multicollinearity and less useful variables through the shrinkage method was more effective compared to the stepwise selection and the Lasso technique.
Overall, the best performing model was the Multivariate K-NN one, with an adjusted $R^2 = 0.627$, by far the highest, and, at the same timing keeping both the error metrics at the lowest. Because of these reasons, the team could assert that the data likely contains non-linear pattern that linear models cannot capture fully; it was then decided that the model to prefer is the non-linear multivariate K-NN Regression Model.

Table 8: Regression models comparison.

Model	Complexity	R^{2}[a]	MAE	RMSE
Linear	Simple	0.499	26.812	44.600
Linear	Multivariate	0.574	24.406	40.548
Linear Ridge	Multivariate	0.585	23.724	39.056
Linear Lasso	Multivariate	0.577	24.272	40.104
Non Linear Knn	Simple	0.504	26.751	43.171
Non Linear Knn	Multivariate	0.627	22.280	36.293
Non Linear Decision Tree	Simple	0.476	27.684	41.815
Non Linear Decision Tree	Multivariate	0.594	23.454	36.741

[a] Adjusted R^2 for multivariate models.

4 Pattern Mining

Pattern Mining is the process of discovering interesting patterns and association rules within the data; considering the relatively small size of the dataset, the team decided to utilize the Apriori algorithm, and did not incur any computational problem. To conduct the analysis, four variables deemed relevant to the target variable, 'Rating', were selected. The *Weight* variable was discretized by rounding values to the nearest integer, resulting in five distinct classes. Similarly, *AveragePlayTime* and *ComAgeRec* were discretized into specific intervals based on domain knowledge:

- *AveragePlayTime*: grouped into ['0-30', '30-60', '60-120', '120+'] minutes.
- *ComAgeRec*: grouped into ['0-6', '7-13', '14-17', '18+'] years.

The *Category* feature was engineered by aggregating the individual binary variables representing each game category. Many board games were not assigned any category, and were excluded from the analysis; for this reason, the final dataset contains just 10447 units.

4.1 Parameters Selection

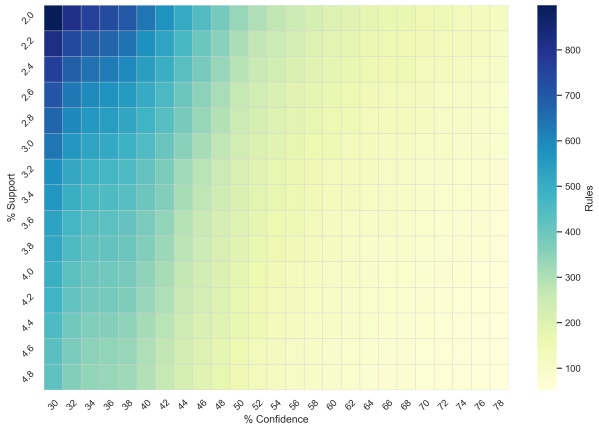
Support To select the minimum support, the team analyzed the behavior of the itemsets in Figure 18b. As shown, there is a massive drop in the number of patterns between 0% and 1%, which suggests the presence of significant noise. Since the curve starts to flatten around the 2% mark, we settled on a threshold of 2.5%. This value strikes a good balance, filtering out irrelevant combinations while keeping a sufficient number of interesting rules. Additionally, as can be observed in Figure 18b, Frequent and Closed itemsets overlap almost perfectly. While there is a significant gap at 0% support (around 1,000 patterns), this difference vanishes almost immediately: even at 0.05%, they differ by only a few units.

Confidence Confidence was chosen together with Support by observing Figure 18b. The difference in Confidence level didn't make too much of a difference, as the Support value was the most impactful parameter of the two; for this reason the value that better fitted with the selected Support was chosen, corresponding to 64%. Confidence is mainly used when determining association rules.

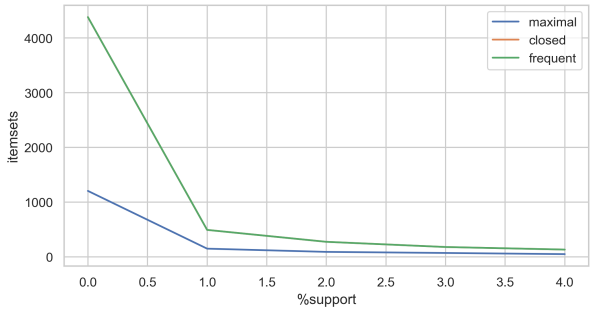
Min Items The minimum number of items to form a pattern was selected as $zmin = 2$; utilizing a $zmin = 1$ was deemed unnecessary, as the Apriori one-on-one rules were too vague.

Figure 18: Parameters Selection

(a) Rules Selection



(b) Support Value



4.2 Pattern Analysis

The Apriori algorithm was used to extract frequent, closed and Maximal itemsets.

4.2.1 Frequent and Closed Itemsets

As it was anticipated in the previous paragraph, the set of Frequent and Closed itemsets are identical with the chosen hyperparameters; the number of itemsets identified by the algorithm is 215. In Table 9, showing the most supported itemsets, the bin "Age 7-13" is always present; there is a great number of games in the dataset that fall inside such range, implying that said age is the most targeted demographic by the board game producers.

Table 9: Top 5 Frequent Itemsets ordered by Support

Itemset	**Support (%)**
{ *Weight 2*, *Age 7-13* }	39.82
{ *Medium Rating*, *Age 7-13* }	36.05
{ *Playtime 0-30*, *Age 7-13* }	31.97
{ *Playtime 30-60*, *Age 7-13* }	25.45
{ *Weight 1*, *Playtime 0-30* }	24.56

4.2.2 Maximal Pattern

Maximal Itemsets are fewer in numbers, counting just 79 sets. The main conceptual difference with Frequent and Closed Itemsets is that the Maximal Itemsets are those extended combinations of characteristics that still make a board game popular; while the other two are more general sets, the Maximal ones identify niche archetypes of games. Observing Table 10, "Age 7-13" is again in most top itemsets. Another popular archetype emerges from the table, easy board games for children (Age 0-6), which corresponds to the "Entry Level" cluster discussed in Paragraph 2.1.2. Other interesting itemsets are mildly difficult strategy games for teenagers and difficult board games with high rating.

Table 10: Top 5 Maximal Itemsets ordered by Support

Maximal Itemset	**Support (%)**	**Length**
{ *Weight 1.0*, *Low Rating*, *Playtime 0-30*, *Age 7-13* }	7.07	4
{ *Weight 1.0*, *Med Rating*, *Playtime 0-30*, *Age 7-13* }	6.48	4
{ *Age 0-6*, *Weight 1.0*, *Low Rating*, *Playtime 0-30* }	5.89	4
{ *Playtime 60-120*, *Weight 2.0*, *Age 7-13* }	5.57	3
{ *High Rating*, *War Games*, *Age 7-13* }	5.26	3

4.3 Association Rules

The Apriori algorithm generated 150 Association Rules, 124 of which with $lift \geq 1$. Afterward, the team tried a few values of Lift to determine the best threshold; it was chosen a value of 1.9 that identifies the 26 most interesting rules. The team tried to analyse the rules that had *Low Rating* and *High Rating* as consequent, to identify the best and worst combinations of characteristics for a board game.
The team focused on the most interesting rules rather than the ones with the higher lift that were deemed too trivial and often characterized by overlapping antecedents. The most interesting rules were reported in Table 11. Observing the rules with *High Rating* as consequent, it becomes clear that the most important aspect of a high rated board game is the complexity (every found rule contains 4.0 Weight as an antecedent): the game should be challenging for the players but not overly obnoxious. On the other hand, rules with *Low rating* as antecedent were harder to discover, and the final set of association rules only contains one of them. It appears that War themed games intended for kids with minimal complexity tend to receive low scores, possibly due to an unsuitable theme for that demographic. It is worth considering that the lift value was way higher for the *High Rating* Association Rules (by around 50%), compared to the *Low Rating* one, making the association between higher ratings and higher complexity more robust. Lastly, an Association Rule for short games was reported in the table. The most frequent pattern appears to be, similar to *Low Rating*, a simple game with a target demographic of kids.

Table 11: Interesting Association Rules

Consequent	**Antecedent**	**Abs. Supp.**	**% Supp.**	**Conf.**	**Lift**
High Rating	{4.0 Weight, 7-13 Age}	233	2.23	0.704	3.010
High Rating	{4.0 Weight, 120+ Playtime}	230	2.20	0.699	2.990
Low Rating	{0-6 Age, War CAT, 1.0 Weight}	208	1.99	0.640	1.961
0-30 Playtime	{0-6 Age, 1.0 Weight}	980	9.38	0.916	2.071